



Progress Report 2

BC Personal Health Management Platform

An AI-Powered Healthcare Navigation and Analytics System

CSIS 4495 - 002: Applied Research Project
Winter 2026

Prepared For:

Padmapriya Arasanipalai Kandhadai

Date: March 2, 2026

Github Link: https://github.com/desporamon/W26_4495_S2_DesmondC

Prepared By:

<u>Name</u>	<u>Student ID</u>	<u>Email Address</u>
Desmond Chua	300369803	chuad1@student.douglascollege.ca

Contents

1. Work Date/Hours Log	2
2. Summary Description of Work Done	5
2.1 Sprint 3A: Home Dashboard Rebuild (Feb 25-27)	5
2.2 Sprint 3B: My Personal Health Dashboard (Feb 28 - Mar 2)	6
2.3 Login Redirect Fix (Mar 2)	8
2.4 Data Verification Methodology	8
2.5 Issues Encountered and Solutions	9
3. Repo Check-in of Implementation Completed	10
4. Revised Timeline and Next Steps	11
4.1 What's Next: Sprints 3C, 3D, and 3E	12
4.2 Exploratory Research: Agentic AI in Healthcare Triage	13
5. AI Use Section	14
5.1 AI Tools Usage Summary	14
5.2 Table of AI Tools and Specific Use	14
5.3 Statement of Originality	15
Appendix A: AI Prompt History	16
Prompt Session 1: Streamlit Multi-Section Dashboard Layout	16
Prompt Session 2: Database Query for Dashboard Statistics	16
Prompt Session 3: Plotly Chart Types for Streamlit Dashboards	17
Prompt Session 4: Plotly Donut Chart Label Clipping	17
Prompt Session 5: Streamlit Page Redirect After Login	18
Prompt Session 6: Claude Code – Dashboard Page Generation	18
Prompt Session 7: Agentic AI Research	19

1. Work Date/Hours Log

Note: Logs are entered as work is completed with granular entries. This reporting period records after cruise break (Feb 14, 22) Sprint 3A (Home Dashboard) and Sprint 3B (Personal Health Dashboard) implementation.

Date	Hours	Description of Work Done
2026-02-25	1.00	Post-cruise return: Reviewed Sprint 3 roadmap tracker, organized project files and mockups into consolidated project workspace
2026-02-25	0.50	Sprint 3A: Studied existing 1_Home.py placeholder code, compared against UI mockup, identified all sections needing rebuild
2026-02-25	1.00	Sprint 3A: Rebuilt Home page Welcome Header section with personalized greeting using session_state username, added health overview subtitle
2026-02-26	0.75	Sprint 3A: Created Quick Actions row (3 styled cards with icons: Check Symptoms, Find Facility, Health Info) with st.page_link() navigation
2026-02-26	1.00	Sprint 3A: Created My Health Summary card querying SQLite database via get_assessment_stats() - displays total assessments, last check relative time, most common symptom with percentage
2026-02-26	0.75	Sprint 3A: Created Recent Activity list showing 3 most recent assessments from database with color-coded urgency dots (green/yellow/red)
2026-02-27	0.50	Sprint 3A: Added BC Health Tip of the Day banner with rotating BC-specific health tips and Find Nearest Pharmacy navigation button
2026-02-27	0.50	Sprint 3A: Added most_common_symptom_count to get_assessment_stats() in database.py for percentage calculation (backward-compatible change)

2026-02-27	0.50	Sprint 3A: Verified all dashboard data against SQLite database using Python queries - confirmed 33 assessments, Chest pain 33%, recent activity entries all match live data
2026-02-27	0.25	Sprint 3A: Git commit - 'Sprint 3A: Rebuild Home Dashboard to match UI mockup'
2026-02-28	0.50	Sprint 3B: Studied My Personal Health Dashboard mockup (2 pages), identified 5 sections: stat cards, trend chart, symptoms breakdown, assessment history table, date filter
2026-02-28	1.50	Sprint 3B: Built dashboard layout with 4 stat summary cards (Total Checks, This Month, Most Common, Avg Urgency) with colored icon boxes and status indicators
2026-03-01	1.50	Sprint 3B: Created Plotly filled area chart for Symptom Trends Over Time - grouped assessments by month, teal color scheme, responsive layout with st.plotly_chart()
2026-03-01	1.00	Sprint 3B: Created Plotly donut pie chart for Symptoms Breakdown - shows distribution of primary symptoms with percentages, rare symptoms grouped as Other
2026-03-01	1.00	Sprint 3B: Created Assessment History HTML table with styled colored badges (green Home Care, yellow Walk-in Clinic, orange Urgent Care, red Emergency)
2026-03-02	0.75	Sprint 3B: Implemented date filter dropdown (All Time, Last 6 Months, Last 3 Months, Last Year) that filters all dashboard sections simultaneously
2026-03-02	0.75	Sprint 3B: Fixed UI issues - Plotly Y-axis tick intervals (dtick=5), donut chart label clipping (moved labels inside slices with legend below), button styling for Export Report and New Assessment

2026-03-02	0.50	Sprint 3B: Automated data verification against SQLite - confirmed all 4 stat cards, trend chart, donut chart percentages, and history table entries match real database values
2026-03-02	0.25	Sprint 3B: Git commit - 'Sprint 3B: Rebuild My Dashboard with Plotly charts, date filter, and live data'
2026-03-02	0.25	Added auto-redirect in app.py to route users to Home Dashboard after login instead of showing old placeholder page; Git commit
2026-03-02	0.75	Learning & Research: Researched agentic AI frameworks in healthcare triage (reviewed Thamma 2025, Wang et al. 2024); Evaluated feasibility for project integration
2026-03-02	0.75	Documentation: Updated work log, prepared Progress Report 2, updated task tracker with Sprint 3A and 3B completion status
TOTAL	16.25	Hours logged for Progress Report 2 (Feb 25 - Mar 2, 2026)

2. Summary Description of Work Done

During this reporting period (February 25 to March 2, 2026), I finalized Sprint 3A (Home Dashboard) and Sprint 3B (My Personal Health Dashboard) of the BC Personal Health Management Platform. This entailed transforming two main pages from mere placeholder contents to fully operational, data-driven interfaces that complied with the finalized UI mockups. The two pages request data from the SQLite database simultaneously, and the values have been checked against the raw database for the confirmation of no use of hardcoded data. Furthermore, a login redirect was set up so that the users after authentication would directly reach the Home Dashboard without opening any other page. In addition, I have started initial studies on agentic AI frameworks as a possible future upgrade of the symptom assessment chatbot.

2.1 Sprint 3A: Home Dashboard Rebuild (Feb 25-27)

The Home page (`pages/1_Home.py`) was totally overhauled to get rid of the generic placeholder content and install a personalized, data-driven dashboard that matches the UI mockup designed in Sprint 1A. The overhaul consisted of four separate parts:

Welcome Header: Shows a personalized greeting by fetching the users name from the Streamlit session_state (e. g. , Welcome back, Test User!) with a subtitle “Here’s your health overview.” This puts the users in the presence of personalization as soon as they log in.

Quick Actions Row: Three visually-striking cards are placed side by side using `st. columns(3)`; each card consists of a colored icon box, title, subtitle, and teal-colored navigation link. The cards route to Check Symptoms (Symptom Assessment page), Find Facility (Facility Finder page), and Health Info (Health Library page) through Streamlits `st. page_link()` to enable smooth navigation.

My Health Summary + Recent Activity: A two-column layout where the left column shows live statistics queried from the SQLite database via `get_assessment_stats()` and `get_user_assessments()`. It displays: Total Assessments count, Last Check as relative time (e. g. 3 days ago), and Most Common symptom with percentage (e. g. Chest pain 33%). The right column shows the 3 most recent assessments with color, coded urgency dots: green for Home Care/Self, Care, yellow for Walk, in Clinic/Routine, and red for Emergency/Urgent. Both columns handle the empty state gracefully for new users with zero assessments.

BC Health Tip Banner A full-width teal banner at the bottom featuring a rotating BC-specific health tip (stable per session using a hash of the date) and a Find Nearest Pharmacy navigation button linking

to the Facility Finder page. Tips include information about HealthLinkBC 811, free flu shots at BC pharmacies, the BounceBack mental health program, and BC PharmaCare eligibility.

Database Enhancement: A small backward-compatible change was introduced in the database.py file that adds `most_common_symptom_count` to the `get_assessment_stats()` return dictionary. Thus, the Home page can do the percentage calculation for the Most Common symptom display locally without making another database query.

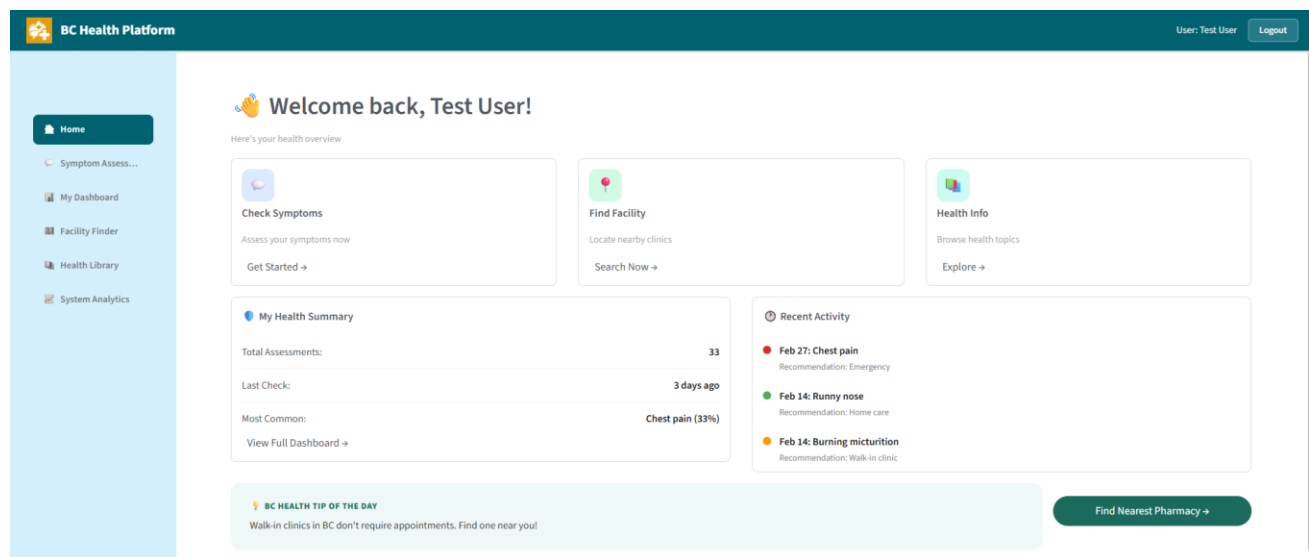


Fig 2.1 – Actual Home Page

2.2 Sprint 3B: My Personal Health Dashboard (Feb 28 - Mar 2)

The My Dashboard page (`pages/3_My_Dashboard.py`) was transformed entirely into a 577-line data visualization dashboard with Plotly charts. This was the single most complicated UI build in Sprint 3, necessitating the incorporation of multiple chart types, a universal date filter, and live queries to the database. The dashboard is broken down into five sections:

Four Summary Stat Cards: Arranged in a row using `st.columns(4)`, each card features a colored icon box with a status indicator dot. Card 1 (blue) shows Total Checks since registration. Card 2 (purple) shows This Month assessment count with the current month name. Card 3 (orange) shows the Most Common symptom with its percentage of total assessments. Card 4 (green) shows Average Urgency, calculated by mapping Emergency=4, Urgent=3, Routine=2, Self-Care=1, averaging all assessments, and mapping back to a label (Critical/High/Moderate/Low).

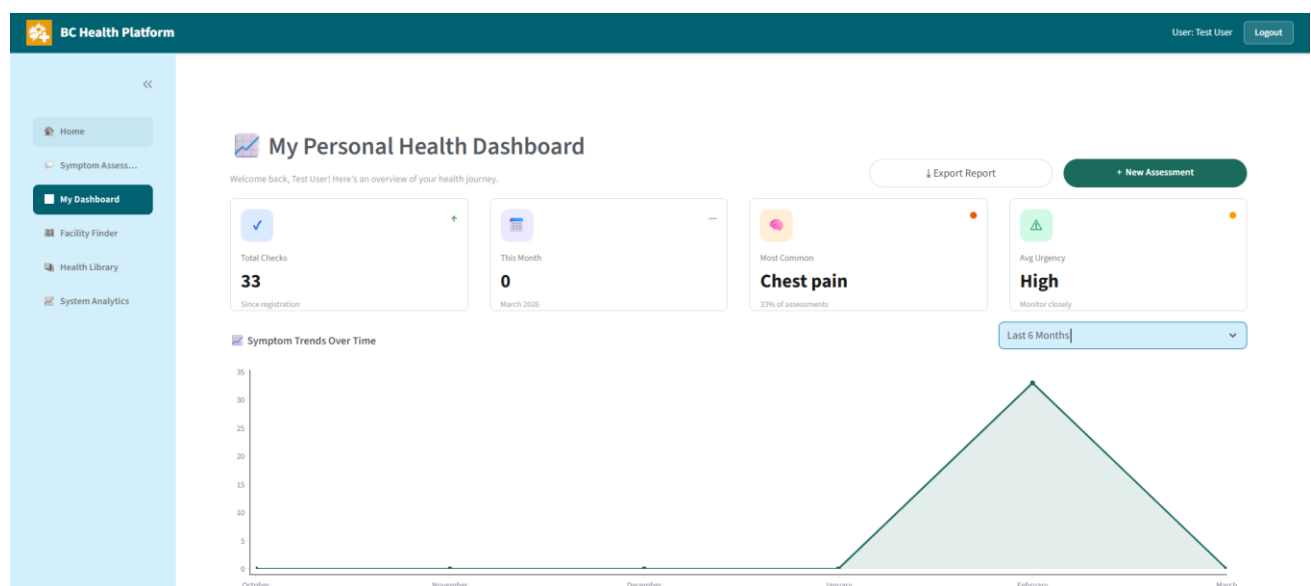
Symptom Trends Over Time: A Plotly filled area chart shows the number of assessments by month. The chart adopts the teal color scheme (#1a6b5c) that goes with the platform theme, is responsive via `st.plotly_chart(use_container_width=True)`, and features neat Y-axis intervals set with `dtick=5` to prevent cluttered labeling.

Symptoms Breakdown: A Plotly donut pie chart illustrating the share of primary symptoms in all assessments. The very first symptom from each assessments extracted_symptoms JSON array is taken. Symptoms that appear only once are combined into an “Other” category. After experimenting with various layout options (outside labels, right, side legend), the final solution employs inside percentage labels with a horizontal legend right below the chart for maximum readability without label cutting.

Assessment History Table: An HTML table shown through `st.markdown()` lists the 5 latest assessments. Each row comprises the date (formatted as “Feb 27, 2026”), the main symptom (capitalized), and a colored result badge/pill. Badge colors correspond to the urgency mapping: green for Home Care, yellow for Walk-in Clinic, orange for Urgent Care, and red for Emergency.

Date Filter: A `st.selectbox` dropdown presents All Time, Last 6 Months, Last 3 Months, and Last Year as options. The filter in effect changes all four stat cards, the trend chart, the donut chart, and the history table. This was done by `session_state` holding the filter value with re-computation of all parts based on the filtered list of assessments.

Additional Features: One button for Export Report triggers a CSV download of all assessment data, while the other for New Assessment leads to the Symptom Assessment page. Each are fashioned as pill-shaped buttons in the page header.



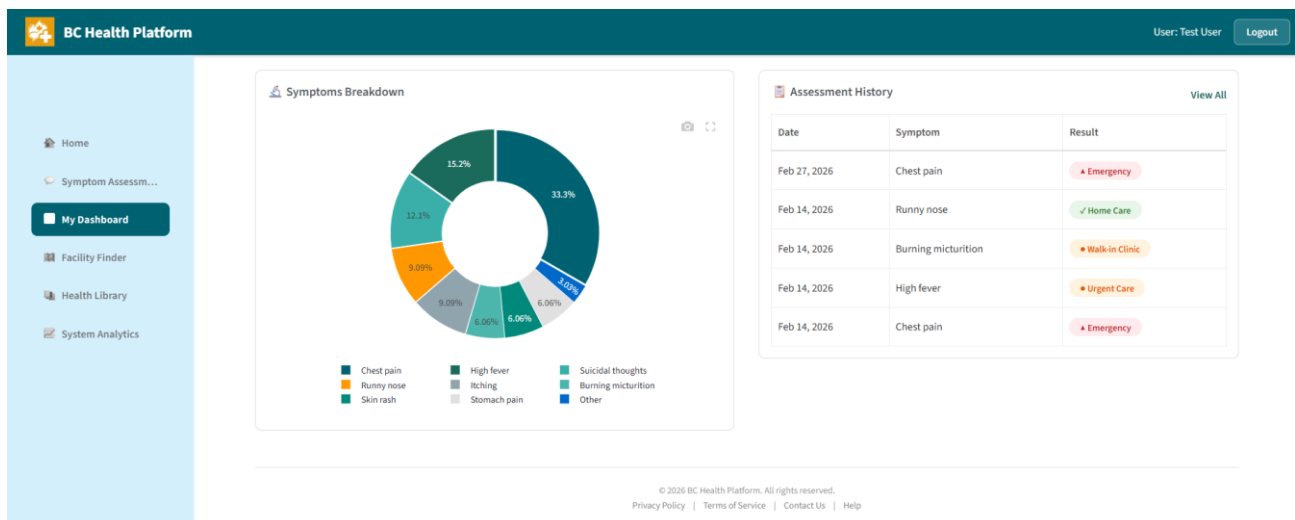


Fig 2.2 – Actual Personal Health Dashboard

2.3 Login Redirect Fix (Mar 2)

A usability adjustment was made in app.py. Earlier, once users logged in, they were taken to the app.py placeholder page that displayed the old content. In order to make the redirection work automatically for authenticated users, a `st.switch_page()` call was inserted to take the logged-in users Home Dashboard (pages/1_Home.py), so that users get the personalized, data-driven interface at once.

2.4 Data Verification Methodology

Both Sprint 3A and 3B implementations were verified against the raw SQLite database to confirm that all displayed values are computed from live data with zero hardcoded values. The verification process involved running Python scripts that import the same database functions used by the pages, executing the same computations, and comparing outputs against what the UI displays. Key verified data points for Test User (id=2):

- Total Assessments: 33 (COUNT(*) from assessments table)
- Most Common Symptom: Chest pain appearing in 11 of 33 assessments (33%)
- Average Urgency: High (raw average 2.82 from 15 Emergency + 5 Urgent + 5 Routine + 8 Self-Care)
- Most Recent Assessment: Feb 27, 2026 - Chest pain - Emergency
- Donut Chart: Chest pain 33.3%, High fever 15.2%, Suicidal thoughts 12.1%, Runny nose 9.09%, Itching 9.09%

2.5 Issues Encountered and Solutions

Issue	Impact	Solution
Plotly donut chart labels clipped on left side and bottom	Symptom names like “Suicidal thoughts” and “Burning micturition” were truncated	Moved percentage labels inside pie slices using <code>textposition='inside'</code> , placed legend below chart with horizontal orientation
Plotly Y-axis showing every integer (0-35)	Symptom Trends chart looked cluttered and unprofessional	Set <code>dtick=5</code> on Y-axis to show clean intervals (0, 5, 10, 15, 20, 25, 30, 35)
Streamlit button CSS overrides not applying	New Assessment button did not match mockup pill-shaped style	Used more specific CSS selectors and fallback selectors for Streamlit’s internal test IDs
SQLite3 CLI not available on Windows	Could not run database verification queries directly	Used Python with <code>sqlite3</code> library to run verification queries programmatically

3. Repo Check-in of Implementation Completed

GitHub Repository: https://github.com/desporamon/W26_4495_S2_DesmondC

The following files have been modified or created since mid-term report submission. All work is committed with descriptive messages and pushed to the main branch:

File/Folder	Description
Implementation/src/pages/1_Home.py	Home Dashboard - complete rebuild with 4 sections (Welcome Header, Quick Actions, Health Summary + Recent Activity, Health Tip Banner)
Implementation/src/pages/3_My_Dashboard.py	Personal Health Dashboard - complete rebuild with Plotly charts, stat cards, date filter, assessment history table (577 lines)
Implementation/src/components/database.py	Added most_common_symptom_count to get_assessment_stats() return dictionary
Implementation/src/app.py	Added auto-redirect to Home Dashboard after successful login

Git Commit History (this reporting period):

- Feb 27: "Sprint 3A: Rebuild Home Dashboard to match UI mockup (completed Feb 27, late commit)"
- Mar 2: "Sprint 3B: Rebuild My Dashboard with Plotly charts, date filter, and live data (completed Mar 2)"
- Mar 2: "Add auto-redirect to Home Dashboard after login in app.py"

4. Revised Timeline and Next Steps

With Sprints 3A and 3B complete, the project is on track for the remaining Sprint 3 features. The cruise break (Feb 14-22) created a gap, but the dashboard implementations were completed efficiently. The revised schedule:

Sprint	Description	Dates	Status	Report
Sprint 0	Environment Setup	Jan 30-31	DONE	PR1
Sprint 1A-E	Foundation (Mockups, DB, Auth, Nav)	Feb 2-5	DONE	PR1
Sprint 2A-D	Symptom Assessment Chatbot (3-Layer Pipeline)	Feb 6-13	DONE	Midterm
Sprint 3A	Home Dashboard (UI rebuild from mockup)	Feb 25-27	DONE	PR2
Sprint 3B	My Dashboard - Personal Analytics (Plotly)	Feb 28 - Mar 2	DONE	PR2
Sprint 3C	Facility Finder	Mar 3-5	Next	PR3
Sprint 3D	Health Education Library	Mar 6-7	Planned	PR3
Sprint 3E	Multi-Turn Chatbot Enhancement	Mar 8-10	Planned	PR3
Sprint 4	System Analytics Dashboard	Mar 11-14	Planned	PR4
Sprint 5	Deployment + Testing	Mar 15-20	Planned	PR5
Sprint 6	Documentation + Final Submission	Mar 21 - Apr 10	Planned	PR5

GANTT CHART - PROJECT TIMELINE													
Phase	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13
	Jan 27	Feb 3	Feb 10	Feb 17	Feb 24	Mar 3	Mar 10	Mar 17	Mar 24	Mar 31	Apr 7	Apr 14	Apr 21
Sprint 0: Environment Setup													
Sprint 1: Basic App Structure													
1A: UI Mockups													
1B: Project Structure													
1C: Database Setup													
1D: Authentication Feature													
1E: Basic Navigation													
Sprint 2: Symptom Chatbot													
2A: Open AI Integration													
2B: Safety Layer - CTAS Rules													
2C: ML Model Training													
2D: Chatbot UI + Integration													
Sprint 3: Features 3A-3D													
3A: Home Dashboard													
3B: My Dashboard - Personal Analytics													
3C: Facility Finder													
3D: Health Education Library													
3E: Multi-Turn Chatbot Enhancement													
Sprint 4: System Analytics													
4A: System Analytics Data													
4B: System Analytics Dashboard													
Sprint 5: Deployment & Testing													
5A: Deployment													
5B: Testing													
Sprint 6: Documentation													

Fig 4 – Gantt Chart

4.1 What's Next: Sprints 3C, 3D, and 3E

The immediate next phase covers the remaining Sprint 3 features:

Sprint 3C - Facility Finder (Mar 3-5): Develop the BC Healthcare Facility Finder page featuring an interactive Folium map, facility information extracted from a CSV file of real BC healthcare facilities, filter dropdowns (type, region, city), emergency contact cards (911, 811, Crisis Line), and facility detail cards showing address, phone, and directions.

Sprint 3D - Health Education Library (Mar 6-7): Develop the Health Education Library page with a JSON data file for 15-20 health education articles. Provide search functionality by keyword, category filter cards (Pain & Injury, Cold & Flu, Medications, Mental Health, etc.), article cards with expandable content, and links to official BC health resources (HealthLinkBC, BCCDC, BC Health Services).

Sprint 3E - Multi-Turn Chatbot Enhancement (Mar 8-10): The symptom assessment chatbot is to be upgraded from a single-turn to a multi-turn conversation. This requires conversation state management via session_state, follow-up question logic (duration, severity, location), updated OpenAI prompts for accumulating symptoms across turns, decision logic for when enough information has been gathered, and UI updates for natural back-and-forth display. This feature is a direct resolution to the single-turn limitation issue shown in the midterm video demo.

4.2 Exploratory Research: Agentic AI in Healthcare Triage

During this reporting period, I began exploring the emerging concept of agentic AI and its potential relevance to this project. Agentic AI refers to autonomous AI systems that can independently plan, execute, and adapt their actions to accomplish complex goals, as opposed to traditional AI that responds to a single input with a single output (Wang et al., 2024). In the healthcare triage domain, recent research has demonstrated promising results: Thamma (2025) presented an agentic AI framework for clinical decision support that achieved 92.4% diagnostic accuracy and 95.2% triage precision by coordinating multiple autonomous AI agents with access to electronic health records and structured memory systems.

This is relevant to my project because the existing 3, layer hybrid classification (OpenAI → CTAS Rules → ML Model) is basically a set pipeline without any autonomous decision-making between layers. Hypothetically, an agentic system would be able to make decisions on its own such as what follow-up questions to ask, when to trigger emergency protocols, and how to adapt its assessment strategy based on the evolving conversation context. So, in place of a fixed multi-turn script, an agentic chatbot could determine its next question in a way that closely corresponds to the risk profile of the patient which is gradually revealed through the dialogue.

However, creating a genuine agentic AI would be a lot more complicated than the current model and therefore would not only need much more research but also more time for development. The basic elements, self-directed planning, tool integration, reflection loops, and persistent memory, have to be designed, tested and thoroughly scrutinized especially if the context is healthcare which is all about safety. With around 1.5 months left till the end of the project, it is out of question to completely integrate agentic AI within the current framework. I am keeping this as a potential next step and will indicate it in the final report as one of the directions in which the capstone project could be developed further.

If time permits toward the end of the project, I may explore a lightweight prototype of agentic behaviour (such as allowing the chatbot to autonomously select its next assessment step), but this would be treated as an optional stretch goal rather than a core deliverable.

References: Wang, L., Ma, C., Feng, X., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345. | Thamma, S.R. (2025). *Agentic AI for Clinical Decision Support: Real-Time Diagnosis, Triage, and Treatment Planning*. *International Journal of Scientific Research in Science, Engineering and Technology*, 12(3), 428–433.

5. AI Use Section

5.1 AI Tools Usage Summary

During Sprint 3A and 3B, AI tools were used sparingly to clarify technical concepts and understand framework-specific patterns. I personally designed all layout specifications based on my UI mockups, selected the chart types and data pipeline logic, designed the urgency scoring algorithm, and created the verification methodology. Claude (Anthropic) was used primarily as a learning aid to understand Streamlit layout patterns, Plotly chart configuration options, and database query approaches—similar to consulting documentation or asking a classmate for clarification.

5.2 Table of AI Tools and Specific Use

AI Tool	Version/Account	Specific Use	Value I Added
Claude (Anthropic)	Claude.ai	Clarifying Streamlit layout patterns for building multi-section dashboards with <code>st.columns()</code> and custom CSS card styling	Designed the complete Home page layout specification myself based on my UI mockup; Claude helped clarify how to combine <code>st.columns</code> with <code>st.markdown</code> for card-style components
Claude (Anthropic)	Claude.ai	Understanding Plotly chart configuration for Streamlit - area charts, donut charts, axis formatting, and responsive sizing	Selected all chart types and designed the data pipeline myself; Claude helped explain Plotly-specific parameters like <code>dtick</code> , <code>textposition</code> , and domain settings
Claude (Anthropic)	Claude.ai	Researching agentic AI concepts and their application to healthcare triage systems	Independently evaluated feasibility for my project; concluded that full agentic implementation exceeds current timeline but documented as future work
Claude (Anthropic)	Claude.ai	Debugging Plotly chart rendering issues - label	Identified each visual issue myself through testing; Claude

		clipping on donut chart and Y-axis interval formatting	helped explain the specific Plotly properties needed to resolve them
Claude (Anthropic)	Claude.ai	Understanding database query patterns for computing dashboard statistics from SQLite assessment data	Designed the urgency scoring algorithm and verification methodology myself; Claude helped clarify how to structure aggregate queries

5.3 Statement of Originality

All layout specifications, database integration requirements, Plotly chart configurations, urgency scoring algorithms, and verification methodology were designed by me. The UI mockups in Sprint 1A were the design source for the two page rebuilds. AI tools were employed to understand how certain Streamlit and Plotly functions work, not to come up with the design or architecture decisions. Claude Code was only used to speed up the writing of some routine code (layout CSS, repetitive card structures) but every output was checked, tested against the SQLite database, and corrected through the iterations where it did not match the mockups. Several iterations were needed for the Plotly charts (Y-axis formatting, donut chart label positioning, chart sizing) which I found through my own testing and instructed the specific fixes. The sprint planning, feature prioritization, and the decision to include the multi-turn chatbot to Progress Report 3 were my independent strategic decisions. The complete prompt history is in the Appendix.

Appendix A: AI Prompt History

The following documents my AI tool interactions during this reporting period. AI was used sparingly to clarify technical concepts and understand framework-specific patterns. Each entry shows the context, my prompt, and the value I contributed beyond the AI response.

Prompt Session 1: Streamlit Multi-Section Dashboard Layout

Date: February 25, 2026 | Tool: Claude (Anthropic)

Context: I had designed the Home Dashboard mockup in Sprint 1A with four sections (Welcome Header, Quick Actions, Health Summary, Recent Activity, Health Tip). I needed to understand how to combine multiple Streamlit layout components (`st.columns`, `st.container`, `st.markdown` with custom CSS) to create card-style UI sections that match my mockup.

My Prompt: *"I'm rebuilding a Streamlit page to match a dashboard mockup. It has a 3-card row at the top, then a 2-column section below with stats on the left and a recent activity list on the right, and a full-width banner at the bottom. How do I structure this layout in Streamlit using `st.columns` and `st.container`? Can I use custom CSS with `st.markdown` to style cards with borders and rounded corners?"*

Summary of Response: Claude explained the pattern of nesting `st.columns()` for horizontal layouts, using `st.container()` for grouping, and injecting custom CSS via `st.markdown(unsafe_allow_html=True)` for card styling. It described how Streamlit's layout system works with its top-to-bottom rendering model.

Value I Added: I designed the entire 4-section layout specification myself based on my mockup. Claude helped me understand the Streamlit components needed to implement it. I then wrote the complete page structure, chose the specific CSS properties (border-radius, box-shadow, padding), selected the teal color scheme (#1a6b5c), and integrated the database queries for live data display.

Prompt Session 2: Database Query for Dashboard Statistics

Date: February 26, 2026 | Tool: Claude (Anthropic)

Context: I needed to display a "Most Common Symptom" with a percentage on the Home Dashboard. The existing `get_assessment_stats()` function in `database.py` returned the most common symptom name but not its count, so I couldn't calculate the percentage.

My Prompt: *"My `get_assessment_stats()` function returns the most common symptom but I need the count as well to calculate a percentage. What's the best way to add a count to the return dictionary without breaking the existing function signature? I want it to be backward-compatible."*

Summary of Response: Claude explained the approach of adding a new key to the return dictionary (which is backward-compatible since existing code only accesses keys it already knows about) and showed how to use SQL's COUNT with GROUP BY and ORDER BY DESC LIMIT 1 to get both the symptom name and its count in a single query.

Value I Added: I identified the specific gap in my existing function. I implemented the change myself in database.py, adding most_common_symptom_count to the return dictionary. I tested the function to ensure it did not break any existing pages that call get_assessment_stats(), and verified the count (11 for Chest pain out of 33 total) against a direct SQL query.

Prompt Session 3: Plotly Chart Types for Streamlit Dashboards

Date: February 28, 2026 | Tool: Claude (Anthropic)

Context: I was designing the My Dashboard page and needed to understand which Plotly chart types would best represent my data. My mockup showed an area chart for trends and a donut chart for symptom distribution, but I had not used Plotly before.

My Prompt: *"I need to create two Plotly charts in Streamlit: (1) a filled area chart showing assessment counts over time grouped by month, and (2) a donut pie chart showing symptom distribution with percentages. How do I configure these in Plotly? Specifically, how do I set the fill under the line for the area chart, and how do I create a donut hole in a pie chart?"*

Summary of Response: Claude explained Plotly's go.Scatter with fill='tozeroy' for area charts and go.Pie with hole parameter for donut charts. It also described how to use st.plotly_chart() with use_container_width=True for responsive rendering in Streamlit.

Value I Added: I selected the chart types based on my mockup design. I designed the complete data transformation pipeline (grouping assessments by month, extracting primary symptoms from JSON arrays, calculating percentages, grouping rare symptoms into "Other"). I chose the teal color scheme to match the platform theme, configured the chart margins and layout, and iterated through three rounds of fixes for the donut chart label positioning.

Prompt Session 4: Plotly Donut Chart Label Clipping

Date: March 2, 2026 | Tool: Claude (Anthropic)

Context: After building the donut chart, I noticed that symptom labels on the left side were being clipped (e.g., "Suicidal thoughts" showed as "houghts") and there were unwanted connector lines. I needed to understand the Plotly properties to fix this.

My Prompt: *"My Plotly donut chart has labels cut off on the left side and weird connector lines. I tried increasing margins but it didn't help. What Plotly properties control where the text labels appear on a pie chart? Can I move them inside the slices instead of outside?"*

Summary of Response: Claude explained the `textposition` parameter ('inside' vs 'outside'), the connector line settings, and the `domain` property for controlling chart size within its container. It also suggested using a horizontal legend below the chart as an alternative to outside labels.

Value I Added: I identified the visual issues through my own testing against the mockup. After understanding the options, I decided on the approach of using inside percentage labels with a horizontal legend below the chart. I tried three different configurations before arriving at the final design that matched my mockup without any label clipping. The decision of which approach to use was mine.

Prompt Session 5: Streamlit Page Redirect After Login

Date: March 2, 2026 | Tool: Claude (Anthropic)

Context: After rebuilding the Home Dashboard in Sprint 3A, I noticed that the `app.py` page still showed the old placeholder content after login. I wanted to automatically redirect users to the new Home page.

My Prompt: *"In my Streamlit app, after a user logs in successfully on `app.py`, they still see the old placeholder page instead of my new Home Dashboard. How do I redirect them to `pages/1_Home.py` after authentication? Is there a Streamlit function for page navigation?"*

Summary of Response: Claude explained Streamlit's `st.switch_page()` function which programmatically navigates to another page. It noted that the function should be called after setting the `session_state` authentication variables.

Value I Added: I identified the usability issue myself during testing. I read `app.py` to understand the authentication flow, determined the exact location where the redirect should be placed (after successful login sets `session_state`), and implemented the one-line fix. The redirect was tested to confirm it works for both new logins and page refreshes.

Prompt Session 6: Claude Code – Dashboard Page Generation

Date: February 25 - March 2, 2026 | Tool: Claude Code (VS Code Extension)

Context: Used Claude Code to accelerate the writing of dashboard page code based on my detailed layout specifications and mockup designs. Two main pages were generated this way.

My Prompt (Home Dashboard): *“Read existing 1_Home.py, database.py, and app.py first to understand session_state variables. Then rebuild 1_Home.py with 4 sections: Welcome Header with session_state username, Quick Actions row with 3 cards linking to other pages, Health Summary + Recent Activity in 2-column layout using get_assessment_stats() and get_user_assessments(), and BC Health Tip banner. Use teal color scheme #1a6b5c.”*

My Prompt (My Dashboard): *“Read existing 3_My_Dashboard.py and database.py. Rebuild with: 4 stat cards (Total Checks, This Month, Most Common, Avg Urgency with scoring Emergency=4 to Self-Care=1), Plotly area chart for monthly trends, Plotly donut chart for symptoms breakdown, HTML table with colored badges for assessment history, and date filter dropdown that updates all sections.”*

Summary of Response: Claude Code generated the page files based on my specifications, reading the existing codebase first to understand the data structures and session variables.

Value I Added: I designed all the layout specifications, selected chart types, defined the urgency scoring algorithm, and specified the database integration patterns. After generation, I reviewed the entire code, verified all data against the SQLite database using Python queries, identified visual issues (donut chart clipping, Y-axis formatting, button styling), and directed multiple iterative fixes. The generated code required several rounds of correction before it matched my mockups.

Prompt Session 7: Agentic AI Research

Date: March 2, 2026 | Tool: Claude (Anthropic)

Context: After completing the dashboard pages, I began researching how emerging AI concepts could further enhance the symptom assessment chatbot beyond the planned multi-turn enhancement.

My Prompt: *“I’ve been reading about agentic AI and autonomous AI agents. How would an agentic approach differ from my current 3-layer pipeline (OpenAI → CTAS Rules → ML Model) for healthcare triage? Is this something I could realistically integrate into my Streamlit project, or is it too complex for a capstone project with 1.5 months remaining?”*

Summary of Response: Claude explained the key differences between pipeline-based AI and agentic AI (autonomous planning, tool use, reflection, memory). It noted that agentic systems in healthcare are still largely experimental and referenced recent literature on multi-agent clinical decision support. It advised that a full agentic implementation would be very ambitious for the remaining timeline.

Value I Added: I independently researched the academic literature (Thamma 2025, Wang et al. 2024) and evaluated the feasibility for my specific project. I concluded that full agentic integration is not feasible within the timeline but documented it as a future enhancement direction. The decision to

include this as exploratory research rather than a committed feature was my strategic choice to maintain realistic project scope.